

CS 598 PSL Fall 2026

Coding Assignment 2

- Anirudh Eswara
- Ryan Lenea
- Mark Synowiec
- Robert Walker

Problem 1 [50 points]

In this problem, we are going to use a simulated example to illustrate the behavior of testing and training errors as the number of predictor increases.

(a) [5 points] Generate a data set with $p = 20$ (independent) covariates, $n = 1,000$ observations and an associated quantitative response vector generated according to the model

$$Y = X\beta + \varepsilon \text{ with}$$

$$\beta = (2, -1, 0, 0.8, 0, -1.5, 1.2, 0, 0, 0.3, -0.7, 0, 1.8, 0, 0.5, 0, -1.2, 0, 0.4, 0)$$

The covariates and the ε can be generated using standard normal random variables. (This is similar to what you did in Coding Assignment # 1).

```
In [1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

RANDOM_SEED = 598
n = 1000

rng = np.random.default_rng(RANDOM_SEED)
X = rng.standard_normal((n, 20))
df = pd.DataFrame(X, columns=[f'X{i}' for i in range(1, 21)])

beta = np.array([2, -1, 0, 0.8, 0, -1.5, 1.2, 0, 0, 0.3, -0.7, 0, 1.8, 0,
0.5, 0, -1.2, 0, 0.4, 0])
eps = rng.standard_normal(n)
y = X @ beta + eps
```

(b) [5 points] Randomly split the data set into a training set containing 200 observations and a test set containing 800 observations.

```
In [2]: # split data int 200 training and 800 test observations
x_train, x_test, y_train, y_test = train_test_split(df, y, train_size=200,
```

```
random_state=RANDOM_SEED)
```

(c) [10 points] Perform best subset selection on the training set, and plot the training set MSE associated with the best model of each size. Recall that

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

```
In [3]: from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

X_tr = x_train.values
X_te = x_test.values

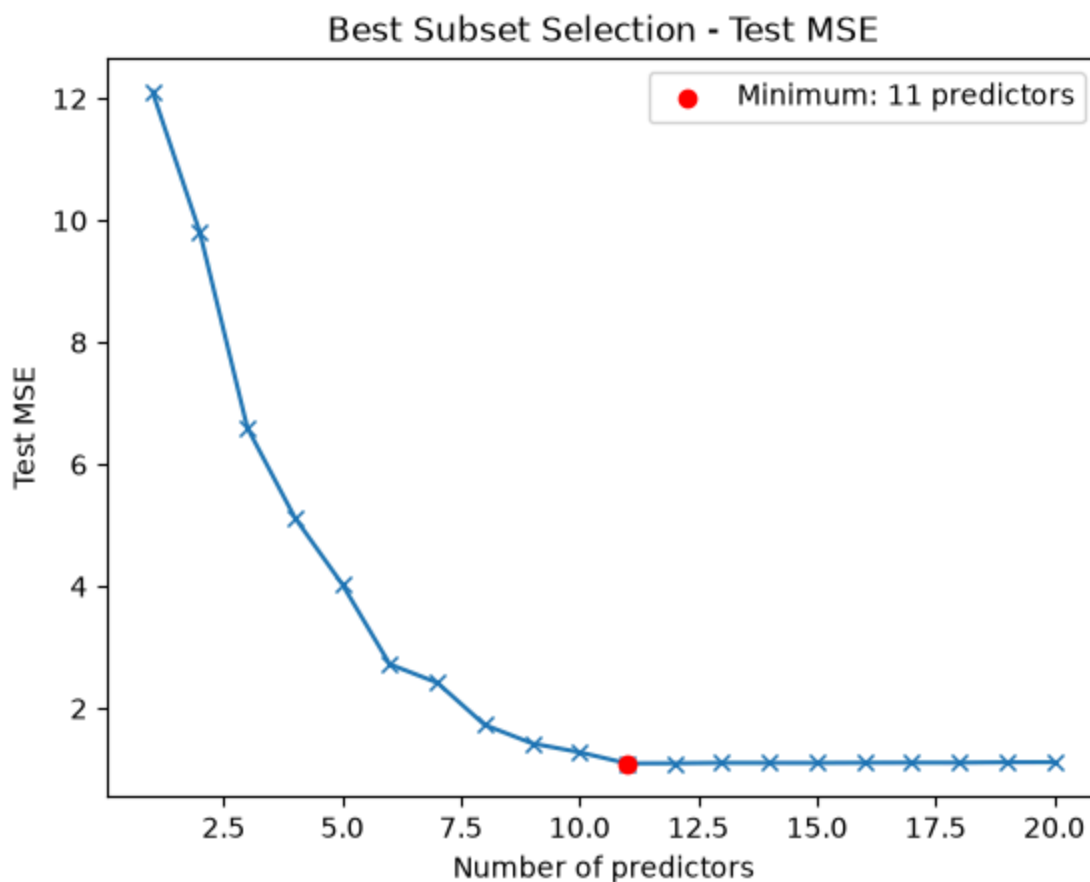
best_models = []
for k in range(1, 21):
    if k == 20:
        cols = list(range(X_tr.shape[1]))
    else:
        selector = SequentialFeatureSelector(
            LinearRegression(),
            n_features_to_select=k,
            direction="forward",
            scoring="neg_mean_squared_error",
            n_jobs=-1,
        ).fit(X_tr, y_train)
        # Which features we should use
        cols = selector.get_support(indices=True).tolist()

    # Actually fit a model with the features determined
    model = LinearRegression().fit(X_tr[:, cols], y_train)
    train_mse = mean_squared_error(y_train, model.predict(X_tr[:, cols]))
    test_mse = mean_squared_error(y_test, model.predict(X_te[:, cols]))
    best_models.append((cols, model, train_mse, test_mse))
```

(d) 10 points] Plot the test set MSE associated with the best model of each size.

```
In [4]: test_mses = [e[3] for e in best_models]
best_size = np.argmin(test_mses) + 1
best_test_mse = test_mses[best_size - 1]

plt.plot(range(1, 21), test_mses, marker="x")
plt.scatter(best_size, best_test_mse, color="red", zorder=3,
            label=f"Minimum: {best_size} predictors")
plt.xlabel("Number of predictors")
plt.ylabel("Test MSE")
plt.title("Best Subset Selection - Test MSE")
plt.legend()
plt.show()
```



(e) [5 points] For which model size does the test set MSE take on its minimum value? Comment on your results.

Remark: If it takes on its minimum value for a model containing only an intercept or a model containing all of the features, then play around with the way that you are generating the data in (a) until you come up with a scenario in which the test set MSE is minimized for an intermediate model size.

```
In [5]: train_mses = [e[2] for e in best_models]
for i, m in enumerate(best_models):
    print(f"For {i + 1} feature(s), train MSE {m[2]} {">=" if m[2] >= m[3]
else "<"} test MSE {m[3]}")
display(best_models[best_size - 1])
```

```

For 1 feature(s), train MSE 13.519168012518936 >= test MSE
12.116339203268282
For 2 feature(s), train MSE 10.290785663612365 >= test MSE 9.803435739052999
For 3 feature(s), train MSE 7.390213430183382 >= test MSE 6.59978191342308
For 4 feature(s), train MSE 5.648593347954984 >= test MSE 5.126624779844722
For 5 feature(s), train MSE 4.055852270782665 >= test MSE 4.032643967836523
For 6 feature(s), train MSE 2.877619905408516 >= test MSE 2.7208531828858837
For 7 feature(s), train MSE 2.127817004201061 < test MSE 2.419496182107642
For 8 feature(s), train MSE 1.5125371170411257 < test MSE 1.7271395427346274
For 9 feature(s), train MSE 1.204258604756896 < test MSE 1.423620685495569
For 10 feature(s), train MSE 1.0676248256907974 < test MSE
1.2782297161468565
For 11 feature(s), train MSE 0.9413033040599441 < test MSE
1.0953046133499087
For 12 feature(s), train MSE 0.9333008994504932 < test MSE
1.0972362515230145
For 13 feature(s), train MSE 0.9295447578972269 < test MSE 1.104218969000584
For 14 feature(s), train MSE 0.9286936185366469 < test MSE
1.1035938436006179
For 15 feature(s), train MSE 0.9285973871216839 < test MSE
1.1030226482464591
For 16 feature(s), train MSE 0.926475345663879 < test MSE 1.1061137908739251
For 17 feature(s), train MSE 0.9262327542244626 < test MSE
1.1077129171345634
For 18 feature(s), train MSE 0.9262247567405685 < test MSE
1.1074027024233235
For 19 feature(s), train MSE 0.9249168406743654 < test MSE
1.1139950155452203
For 20 feature(s), train MSE 0.9244543003228989 < test MSE
1.1151684093205965
([0, 1, 3, 5, 6, 9, 10, 12, 14, 16, 18],
 LinearRegression(),
 0.9413033040599441,
 1.0953046133499087)

```

Comments on results:

For subsets of 6 features and less, the test set MSE was less than the training set MSE, though the test set MSE was minimized at 11 features. The training set MSE, however, was minimized only when using all available features in the dataset. The MSE was high for both with few features because the model was underfitted. This correlates with how the data was generated because out of the 20 initial random features, only 11 were used to generate y . At the top end where the model used all 20 features, the training set MSE being lower than the test set MSE, while the test set MSE increases past 11, shows that the model was overfitted to the training data. Past the optimal number of features, 11, the training set MSE shows a drastically scaled-down reduction in MSE, showing diminishing gains due to incorporation of noise from features unrelated to y .

(f) [5 points] How does the model at which the test set MSE is minimized compare to the true model used to generate the data? Comment on the coefficient values.

```
In [6]: beta_dist = []
for i, m in enumerate(best_models):
    cols = m[0]
    beta_hat_full = np.zeros(len(beta))
    beta_hat_full[cols] = m[1].coef_
    beta_dist.append(float(np.linalg.norm(beta - beta_hat_full)))

best_cols = best_models[best_size - 1][0]
print(f"Best model ({best_size} features): {best_cols}")
print("True betas:      ", beta[best_cols])
print("Estimated betas:", best_models[best_size - 1][1].coef_)
print(f"Euclidean distance to true beta: {beta_dist[best_size - 1]:.4f}")
```

```
Best model (11 features): [0, 1, 3, 5, 6, 9, 10, 12, 14, 16, 18]
True betas:      [ 2. -1.   0.8 -1.5  1.2  0.3 -0.7  1.8  0.5 -1.2  0.4]
Estimated betas: [ 2.01017839 -1.16857305  0.84923028 -1.46673176  1.2344963
 0.37059289
-0.83833733  1.87261263  0.53931703 -1.31201144  0.34765297]
Euclidean distance to true beta: 0.2819
```

Comments on MSE:

The resultant $\hat{\beta}$'s closely resembled the β 's used to generate y . Interestingly, as features were added up to 11, only features used to initially generate y were consecutively added, meaning up until the optimal feature set, only meaningful features were selected. This makes sense since the features that correlate with y would have the biggest impact on minimizing the test MSE. Also, while not completely consistent, the features first selected below the optimal 11 features tended to be those with the largest magnitude of beta when generating y . So for a single feature, feature 0 was selected with a beta of 2.0, followed by feature 5 which was added, with a beta of -1.5. This general trend continued, with the last feature added at the 11 feature set being 18 which had one of the smallest non-zero beta magnitudes (0.4).

This exercise highlighted the importance and impact of relevant features vs the side effect of irrelevant features decreasing performance with the test set.

(g) [10 points] Create a plot displaying

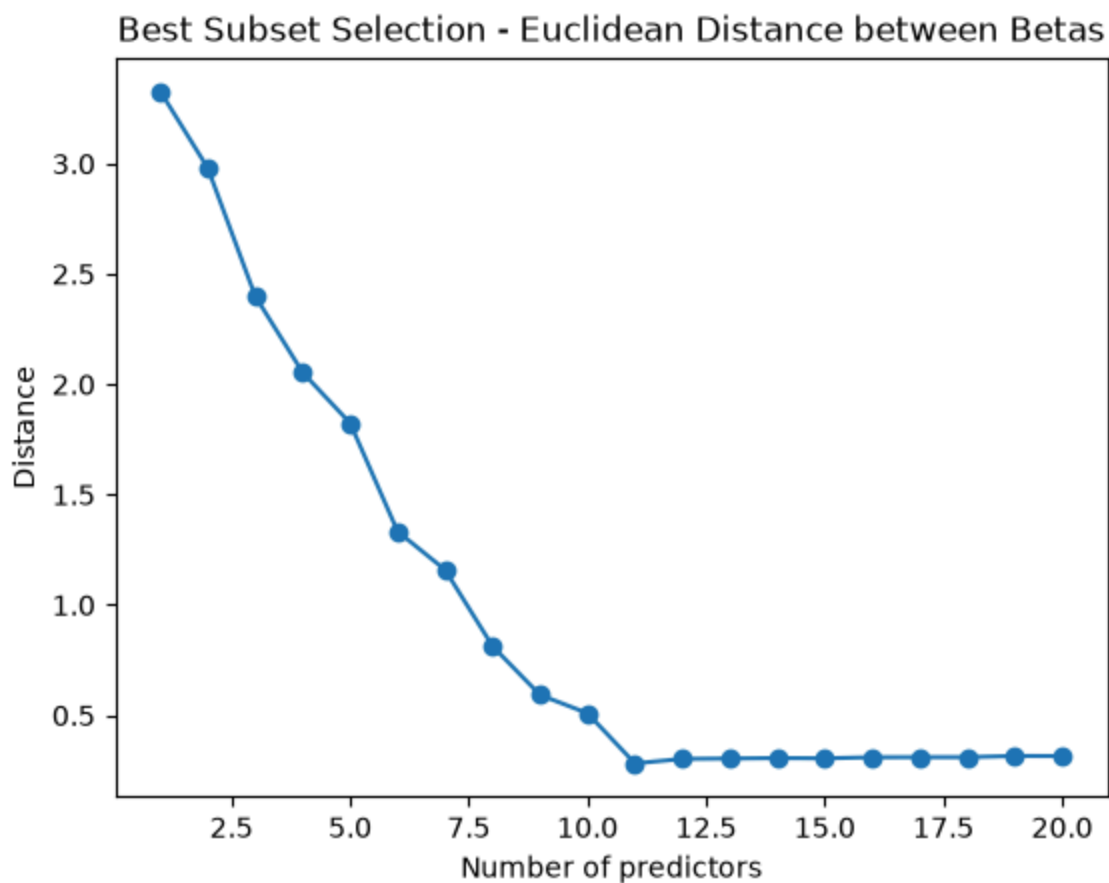
$$\sqrt{\sum_{i=1}^n (\beta_j - \hat{\beta}_j^r)^2}$$

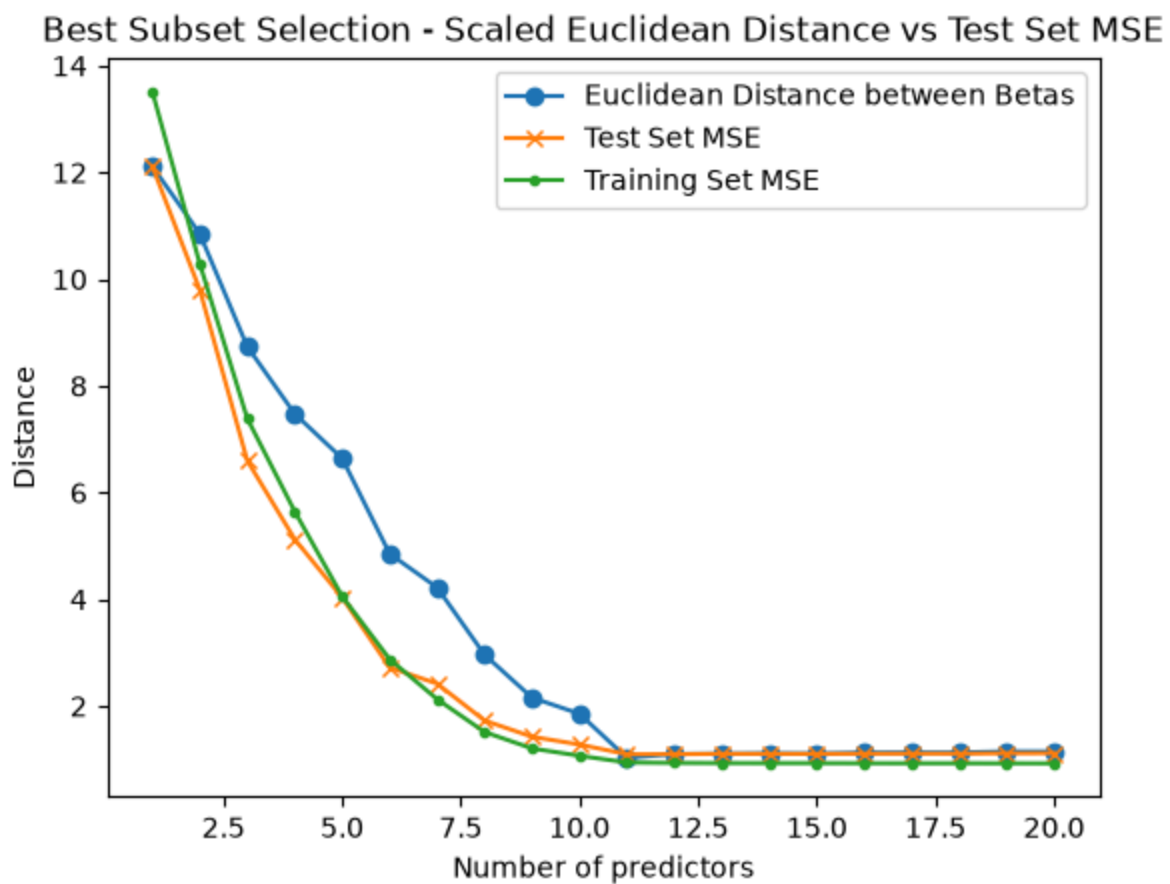
for a range of values of r , where $\hat{\beta}_j^r$ is the j th coefficient estimate for the best model containing r coefficients. Comment on what you observe. How does this compare to the test MSE plot from (d)?

```
In [7]: plt.plot(range(1, 21), beta_dist, marker="o")
plt.xlabel("Number of predictors")
plt.ylabel("Distance")
plt.title("Best Subset Selection - Euclidean Distance between Betas")
```

```
plt.show()

plt.plot(range(1, 21), np.multiply(beta_dist, test_mses[0]/beta_dist[0]),
marker="o", label="Euclidean Distance between Betas")
plt.plot(range(1, 21), test_mses, marker="x", label="Test Set MSE")
plt.plot(range(1, 21), train_mses, marker=".", label="Training Set MSE")
plt.xlabel("Number of predictors")
plt.ylabel("Distance")
plt.title("Best Subset Selection - Scaled Euclidean Distance vs Test Set MSE")
plt.legend()
plt.show()
```





Comments on observations:

The minimum euclidean distance between the betas of the model occurs on the same model that minimized the test set MSE. This value specifically describes how different the coefficients used to estimate y are from the actual coefficients used to generate y . It doesn't take into account what values are in the training set or the test set, though if it were possible to know the truth model in a real world scenario (which it isn't) it could give a sense for how accurate a model is. There wouldn't be a real world situation where a model would be preferred over using an exact formula to calculate expected values rather than use predictions outside of learning or research, but it's interesting to understand the implications of what this distance means for what a model is meant to accomplish. If there does exist an exact set of coefficients that correlate observations with an outcome, these models are designed to formulate that equation based only on observed data.

The second plot above shows the Beta Euclidean Distance scaled to match the first value of the Test Set MSE along with both sets of MSEs. The shape of the distance and test set MSE curves are similar, though below the optimal model, the beta euclidean distance drops quicker than the test set MSE. The drop in the test set MSE is more linear since the model was trained on a different data set - the training data set. The curve showing the training set MSE more closely aligns with the beta euclidean distance since the $\hat{\beta}$ values were generated from the training set.

Problem 2 [50 points – 10 points each]

We will use the [Parkinsons Telemonitoring data set](#) from the UCI Machine Learning Repository.

The data contain 5,875 biomedical voice measurements from 42 patients with early-stage Parkinson's disease. The goal is to predict a textitpatient's disease severity (UPDRS score) from a collection of voice measures and a few demographic variables.

You should use the following variables as predictors:

Variable	Description
age	Subject age
sex	Subject gender (0 = male, 1 = female)
test-time	Days since recruitment into the trial
Jitter(%), Jitter(Abs) Jitter:RAP, Jitter:PPQ5, Jitter:DDP	Measures of variation in fundamental frequency
Shimmer, Shimmer(dB), Shimmer:APQ3	Measures of variation in amplitude

Variable	Description
Shimmer:APQ5, Shimmer:APQ11, Shimmer:DDA	
NHR, HNR	Noise-to-tonal component ratios
RPDE	Nonlinear dynamical complexity measure
DFA	Signal fractal scaling exponent
PPE	Nonlinear measure of fundamental frequency variation

Response: total UPDRS – clinician’s total UPDRS score, linearly interpolated.

Data preparation: Download the data, remove the columns `subject#` and `motor_UPDRS`, and create a random 70 % / 30 % train–test split. Set the random seed to 598.

```
In [8]: # first, pip install ucimlrepo

from ucimlrepo import fetch_ucirepo
import sklearn
import pandas as pd

# fetch dataset
parkinsons_telemonitoring = fetch_ucirepo(id=189)

# clean dataset
# parkinsons_telemonitoring =

# data (as pandas dataframes)
X = parkinsons_telemonitoring.data.features
y = parkinsons_telemonitoring.data.targets

# # metadata
# print(parkinsons_telemonitoring.metadata)

# # variable information
# print(parkinsons_telemonitoring.variables)

# data prep
y = y.drop(columns="motor_UPDRS")
X_train, X_test, y_train, y_test =
sklearn.model_selection.train_test_split(X, y, test_size=0.3,
random_state=598)
```

(a) Fit a best-subset selection algorithm (up to 8 predictors, excluding the intercept) and report the best model of each size together with its training and test prediction errors (MSE). Keep the output concise.

```
In [9]: import numpy
import itertools
```

```

from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression
from math import inf

best_subsets = []

for k in range(1,9):
    lowest_train_mse_k = inf
    lowest_test_mse_k = inf
    lowest_rss_k = inf

    for x in itertools.combinations(X, k):

        train_subset = X_train[list(x)]
        test_subset = X_test[list(x)]

        ### TRAINING
        lr = LinearRegression()
        lr.fit(train_subset, y_train)
        y_pred_train = lr.predict(train_subset)

        # current model mse/rss
        train_mse = mean_squared_error(y_train, y_pred_train)
        n = len(y_train)
        rss_k = n * train_mse

        ### TESTING
        y_pred_test = lr.predict(test_subset)
        test_mse = mean_squared_error(y_test, y_pred_test)

        # rss_k minimized on training error
        if(rss_k < lowest_rss_k):
            lowest_rss_k = rss_k
            lowest_train_mse_k = train_mse
            lowest_test_mse_k = test_mse
            best_cols_k = x

        # report best model of each size with training and testing MSE
        print("Best model of size", k)
        print("Model columns:", best_cols_k)
        print("Training MSE:", lowest_train_mse_k)
        print("Testing MSE:", lowest_test_mse_k)
        print("\n")
        best_subsets.append([k, lowest_rss_k, best_cols_k, lowest_train_mse_k,
lowest_test_mse_k])

```

Best model of size 1
Model columns: ('age',)
Training MSE: 101.2319847635533
Testing MSE: 108.67337172014066

Best model of size 2
Model columns: ('age', 'HNR')
Training MSE: 99.17069173639733
Testing MSE: 107.01184520386897

Best model of size 3
Model columns: ('age', 'HNR', 'DFA')
Training MSE: 97.3324058295774
Testing MSE: 104.7757613154166

Best model of size 4
Model columns: ('age', 'Shimmer:DDA', 'HNR', 'DFA')
Training MSE: 95.97636865941816
Testing MSE: 103.67306427184784

Best model of size 5
Model columns: ('age', 'Shimmer:DDA', 'HNR', 'DFA', 'sex')
Training MSE: 94.90695033020255
Testing MSE: 102.00424768329167

Best model of size 6
Model columns: ('age', 'test_time', 'Shimmer:DDA', 'HNR', 'DFA', 'sex')
Training MSE: 94.40537269884128
Testing MSE: 100.85206067362543

Best model of size 7
Model columns: ('age', 'test_time', 'Shimmer:APQ5', 'Shimmer:APQ11', 'HNR', 'DFA', 'sex')
Training MSE: 94.13483569165545
Testing MSE: 100.12559480683353

Best model of size 8
Model columns: ('age', 'test_time', 'Jitter:DDP', 'Shimmer:DDA', 'HNR', 'HNR', 'DFA', 'sex')
Training MSE: 93.86114705237662
Testing MSE: 100.64885309382574

(b) Using the models from (a), which model is preferred by (i) AIC, (ii) BIC, (iii) Mallows' C_p , and (iv) adjusted R^2 ? For each of these four chosen models report both the training and the test MSE.

```

In [10]: from math import log, inf

# Mallow requires sigma squared
lr_full = LinearRegression()
lr_full.fit(X_train, y_train)
y_pred_full = lr_full.predict(X_train)

full_mse = mean_squared_error(y_train, y_pred_full)
rss_full = n * full_mse
p_full = len(X_train.columns)
sigma2_mallow = rss_full / (n - p_full - 1)

# Adjusted-R-Squared requires Total Sum of Squares (TSS)
tss = ((y_train - y_train.mean()) ** 2).sum()["total_UPDRS"]

# Finding minimum and maximum values
min_aic = inf
min_bic = inf
min_c_p = inf
max_adj_r_squared = -inf

# Search through each model found in 2(a)
for row in best_subsets:

    # save variables
    k = row[0]
    p = row[0]
    rss = row[1]

    # AIC
    aic = (n * log(rss / n)) + (2 * p)
    # BIC
    bic = (n * log(rss / n)) + (log(n) * p)
    # Mallow
    c_p = (rss / sigma2_mallow) + (2 * p) - n
    # Adjusted-R-Squared
    sigma2_r_squared = rss / (n - p - 1)
    sigma2_0_r_squared = tss / (n - 1)
    adj_r_squared = 1 - (sigma2_r_squared / sigma2_0_r_squared)

    if aic < min_aic:
        min_aic = aic
        min_aic_model = row
    if bic < min_bic:
        min_bic = bic
        min_bic_model = row
    if c_p < min_c_p:
        min_c_p = c_p
        min_c_p_model = row
    if adj_r_squared > max_adj_r_squared:
        max_adj_r_squared = adj_r_squared
        max_adj_r_squared_model = row

# report training and test MSE for each best
print("Model preferred by AIC")

```

```

print("Model size:", min_aic_model[0]) # k
print("Model columns:", min_aic_model[2]) # cols
print("Training MSE:", min_aic_model[3]) # train MSE
print("Testing MSE: ", min_aic_model[4]) # test MSE
print("\n")

print("Model preferred by BIC")
print("Model size:", min_bic_model[0]) # k
print("Model columns:", min_bic_model[2]) # cols
print("Training MSE:", min_bic_model[3]) # train MSE
print("Testing MSE: ", min_bic_model[4]) # test MSE
print("\n")

print("Model preferred by Mallow C_p")
print("Model size:", min_c_p_model[0]) # k
print("Model columns:", min_c_p_model[2]) # cols
print("Training MSE:", min_c_p_model[3]) # train MSE
print("Testing MSE: ", min_c_p_model[4]) # test MSE
print("\n")

print("Model preferred by Adjusted-R-squared")
print("Model size:", max_adj_r_squared_model[0]) # k
print("Model columns:", max_adj_r_squared_model[2]) # cols
print("Training MSE:", max_adj_r_squared_model[3]) # train MSE
print("Testing MSE: ", max_adj_r_squared_model[4]) # test MSE

```

Model preferred by AIC

Model size: 8

Model columns: ('age', 'test_time', 'Jitter:DDP', 'Shimmer:DDA', 'NHR', 'HNR', 'DFA', 'sex')

Training MSE: 93.86114705237662

Testing MSE: 100.64885309382574

Model preferred by BIC

Model size: 8

Model columns: ('age', 'test_time', 'Jitter:DDP', 'Shimmer:DDA', 'NHR', 'HNR', 'DFA', 'sex')

Training MSE: 93.86114705237662

Testing MSE: 100.64885309382574

Model preferred by Mallow C_p

Model size: 8

Model columns: ('age', 'test_time', 'Jitter:DDP', 'Shimmer:DDA', 'NHR', 'HNR', 'DFA', 'sex')

Training MSE: 93.86114705237662

Testing MSE: 100.64885309382574

Model preferred by Adjusted-R-squared

Model size: 8

Model columns: ('age', 'test_time', 'Jitter:DDP', 'Shimmer:DDA', 'NHR', 'HNR', 'DFA', 'sex')

Training MSE: 93.86114705237662

Testing MSE: 100.64885309382574

All methods, AIC, BIC, Mallow's c_p , and Adjusted-R-Squared, prefer the model of size 8 with columns: ('age', 'test_time', 'Jitter:DDP', 'Shimmer:DDA', 'NHR', 'HNR', 'DFA', 'sex')

(c) Fit a ridge-regression model to predict `total_UPDRS`. Use cross-validation to select the regularization parameter λ .

```
In [11]: from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
y_train = y_train.squeeze()

alphas = [1e-3, 1e-2, 1e-1, 1, 10, 100, 1000]
ridge_regression_model = RidgeCV(alphas=alphas).fit(X_train_scaled, y_train)
print("Regularization Parameter:", ridge_regression_model.alpha_)
pred_total_UPDRS_ridge_regression_model =
ridge_regression_model.predict(X_test_scaled)
```

Regularization Parameter: 100.0

(d) Fit a lasso model on the same data. Identify which features are driven to exactly zero.

```
In [12]: from sklearn.linear_model import LassoCV

lasso_model = LassoCV(max_iter=100000, alphas=alphas, tol=1e-
3).fit(X_train_scaled, y_train)
print("Regularization Parameter:", lasso_model.alpha_)
pred_total_UPDRS_lasso_model = lasso_model.predict(X_test_scaled)

zero_features = X_train.columns[lasso_model.coef_ == 0][0] #[0] is cleanup
code - Jitter is only feature driven to 0
print("Features Driven to Exactly Zero:", zero_features)
```

Regularization Parameter: 0.01

Features Driven to Exactly Zero: Jitter(%)

(e) Compare the predictive performance (test MSE) of the models selected in (b) with ridge and with lasso. Which approach would you recommend for predicting total UPDRS and why?

```
In [13]: test_mse_ridge_regression_model = mean_squared_error(y_test,
pred_total_UPDRS_ridge_regression_model)
test_mse_lasso_model = mean_squared_error(y_test,
pred_total_UPDRS_lasso_model)
print(ridge_regression_model.coef_)
print(lasso_model.coef_)

print("Test MSE for Ridge-Regression Model:",
test_mse_ridge_regression_model)
print("Test MSE for Lasso Model:", test_mse_lasso_model)
```

```
print("Test MSE for Model preferred by AIC:", min_aic_model[4])
print("Test MSE for Model preferred by BIC:", min_bic_model[4])
print("Test MSE for Model preferred by Mallow C_p:", min_c_p_model[4])
print("Test MSE for Model preferred by Adjusted-R-squared:",
max_adj_r_squared_model[4])
```

```
[ 2.63942738  0.71679575  0.17110131 -1.58434165  0.89183247 -0.61535432
 0.89235666  0.29184623 -0.91207951 -0.56219421 -0.43577814  0.94009829
-0.56259275 -0.68190322 -2.57908317  0.53901532 -2.01189377  1.08692967
-1.08679371]
[ 2.68378804  0.73514623  0.          -1.82295823  1.46346396 -0.9879105
 0.92532109  0.95386758 -1.83998946 -0.67425309 -0.20577923  1.0478327
-0.65900209 -0.62511628 -2.83626139  0.47570495 -2.09101717  1.18316156
-1.16611811]
```

Test MSE for Ridge-Regression Model: 98.56615091853413

Test MSE for Lasso Model: 98.52246456413644

Test MSE for Model preferred by AIC: 100.64885309382574

Test MSE for Model preferred by BIC: 100.64885309382574

Test MSE for Model preferred by Mallow C_p: 100.64885309382574

Test MSE for Model preferred by Adjusted-R-squared: 100.64885309382574

In (b), the same model was selected as the best model according to AIC, BIC, Mallows' C_p , and adjusted R^2 . I would recommend the model selected in (b) over the ridge-regression and lasso models because it uses only 8 features, compared with 19 features in the ridge-regression model and 18 features in the lasso model after one coefficient was driven to 0. Although the ridge-regression and lasso models have lower test MSEs than the model selected in (b), the small improvement in predictive performance does not justify the additional complexity of including 10 or 11 extra features.